

2.13 - Arquitetura hexagonal

Separando domínio de infraestrutura

O problema do acoplamento com infraestrutura

Quando o detalhe técnico invade a regra de negócio

- **Lógica de negócio chama infraestrutura diretamente**
 - Banco, API externa e fila espalhados pelo código
- **Trocar um provedor vira caçada por todo o sistema**
 - Cada ponto de chamada é um risco de regressão
- **Testar o domínio exige subir infraestrutura real**
 - Testes ficam lentos, frágeis e dependentes de rede
- **A regra de negócio se mistura com detalhe técnico**
 - O que deveria durar anos muda junto com o efêmero

O domínio depende da porta, não do detalhe

TYPESCRIPT

```
// Port: o dominio define o contrato que precisa
interface ProvedorDeRotas {
  calcular(origem: Local, destino: Local): Promise<Rota>;
}

// O dominio so conhece a porta, nunca o provedor
class ServicoDeEntrega {
  constructor(private rotas: ProvedorDeRotas) {}
}

// Adapter: a implementacao concreta fica do lado de fora
class AdapterHereMaps implements ProvedorDeRotas { /* ... */ }
```

Ports e adapters: a fronteira explícita

- **Port é a interface que o domínio define**
 - Descreve o que precisa, não como é feito
- **Adapter implementa a port para um detalhe real**
 - Postgres, HERE Maps ou Kafka entram como adapters
- **O domínio não conhece nenhuma tecnologia concreta**
 - A dependência aponta para dentro, nunca para fora
- **Trocar de provedor é escrever um novo adapter**
 - A interface permanece, o resto do sistema nem percebe

Hexagonal torna o domínio testável

- Com ports, o domínio testa contra implementações falsas
- Sem banco, sem rede, sem infraestrutura no teste

A maior recompensa da arquitetura hexagonal não é trocar de provedor, é testar a regra de negócio em milissegundos, com adapters de mentira, sem depender de nada externo.

- Domínio puro é domínio rápido de testar e evoluir

Trocar Google Maps por HERE: 23 pontos sem abstração

- **Trocar de provedor de mapas reduziria custo em 60%**
 - API do Google chamada direto em 23 pontos do código

LUNALOG

A LunaLog precisou trocar o provedor de mapas do Google Maps para o HERE Maps, uma redução de custo de 60%. O problema: o cálculo de rotas chamava a API do Google diretamente em 23 pontos diferentes da aplicação, sem nenhuma abstração. A troca levou seis semanas e introduziu três bugs de regressão. Com arquitetura hexagonal e um adapter por trás de uma interface, essa mudança seria uma implementação nova atrás de um contrato existente. Dias, não semanas.



A arquitetura hexagonal separa domínio de infraestrutura com elegância. Mas e quando o próprio estado precisa ser dividido entre quem lê e quem escreve? Ou quando o histórico de como o sistema chegou ao estado atual vale mais do que o estado em si? Existem dois padrões que mudam a forma de pensar sobre dados e tempo.

CQRS e Event Sourcing